HSLU Lucerne University of Applied Sciences and Arts

Data Pipeline with Apache Airflow

DENG
Forooz Shahbazi

Information Technology

FH Zentralschweiz

Fun Facts and more about Airflow!

- Maxime Beauchemin kicked off the Airflow project at Airbnb in 2014.
- Airflow was developed from the beginning as a noncommercial open source project.
- Airflow open source project is extremely active, with a high rate of commits and a quick response time for bugs and security issues.
- Airflow is available commercially as a managed service or software distribution through many vendors, including GCP, AWS, and Astronomer.io.

Source

Apache Airflow: Introduction

- Open-source workflow orchestration platform
- Author, schedule, and monitor workflows
- Workflows represented as Directed Acyclic Graphs (DAGs)
- A DAG is made of tasks which are arranged in a specific order to be executed.

Core Components

Scheduler

Triggers tasks from DAGs

Executor

Assigns tasks to workers

Workers

Run the tasks

Web Server

Graphical UI

Metadata Database

Stores DAG/task state

DAG Directory

Workflow definition files

DAGs & Tasks

DAG : Directed Acyclic Graph
>>
workflow of tasks

Each task is a unit of work (ingestion, analysis, reporting)

Example tasks: data ingestion, analysis, saving, reporting, error alerts

Task Lifecycle (States)

- No Status → not yet queued
- Scheduled → dependencies met
- Queued → waiting for worker
- Running → task in progress
- Success → completed successfully
- Failed → error occurred
- Up for Retry → the task is scheduled according to the retrial conditions

Apache Airflow Features

- Pure Python → define workflows flexibly
- Useful UI → monitor, schedule, debug workflows
- Integrations → many plug-and-play connectors
- Open Source → strong community contributions
- Ease of use → DAGs with operators & sensors

Apache Airflow Principles

Scalable → modular, message queues, infinite scaling

Dynamic → Python DAGs allow complex workflows

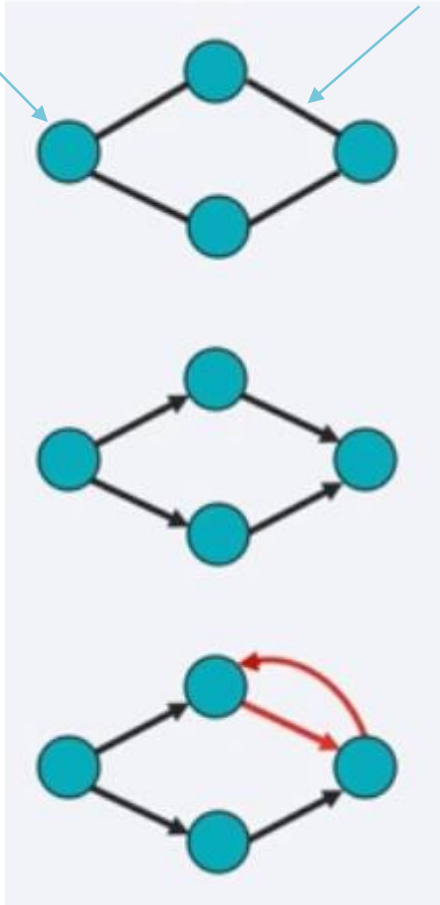
Extensible → custom operators & sensors

What is a DAG?

- Directed Acyclic Graph (DAG)
- Nodes = tasks, Edges = dependencies
- Directed → arrows show execution order
- Acyclic → no loops allowed
- Workflows as graphs in code
- DAGs are defined as python scripts.
- Each task implements an operator for example Python, SQL or bash operator.

Nodes are tasks and edges are dependencies.

Edges define the order in which two tasks should run.



Components of a DAG File

```python
from datetime import timedelta
from airflow import DAG
from airflow.utils.dates import days_ago
from airflow.operators.bash import BashOperator

default_args = {
    "owner": "me",
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    dag_id="simple_tasks",
    description="Two simple Bash tasks chained: task_1 -> task_2",
    default_args=default_args,
    start_date=days_ago(1),       # not in the future
    schedule_interval="* * * * *",# every minute (change as you like)
    catchup=False,
    tags=["example"],
) as dag:
```

Library Imports

DAG arguments (owner, retries, etc.)

DAG definition (id, schedule, description)

Components of a DAG File

- Task definitions (Bash, Python, SQL operators, etc.)
- Task pipeline (dependencies)

```
) as dag:

    task_1 = BashOperator(
        task_id="task_1",
        bash_command='echo "Running TASK 1: $(date)"'
    )

    task_2 = BashOperator(
        task_id="task_2",
        bash_command='echo "Running TASK 2 after TASK 1: $(date)"'
    )

    # Order: task_1 must finish before task_2 starts
    task_1 >> task_2
```

Apache Airflow Scheduler

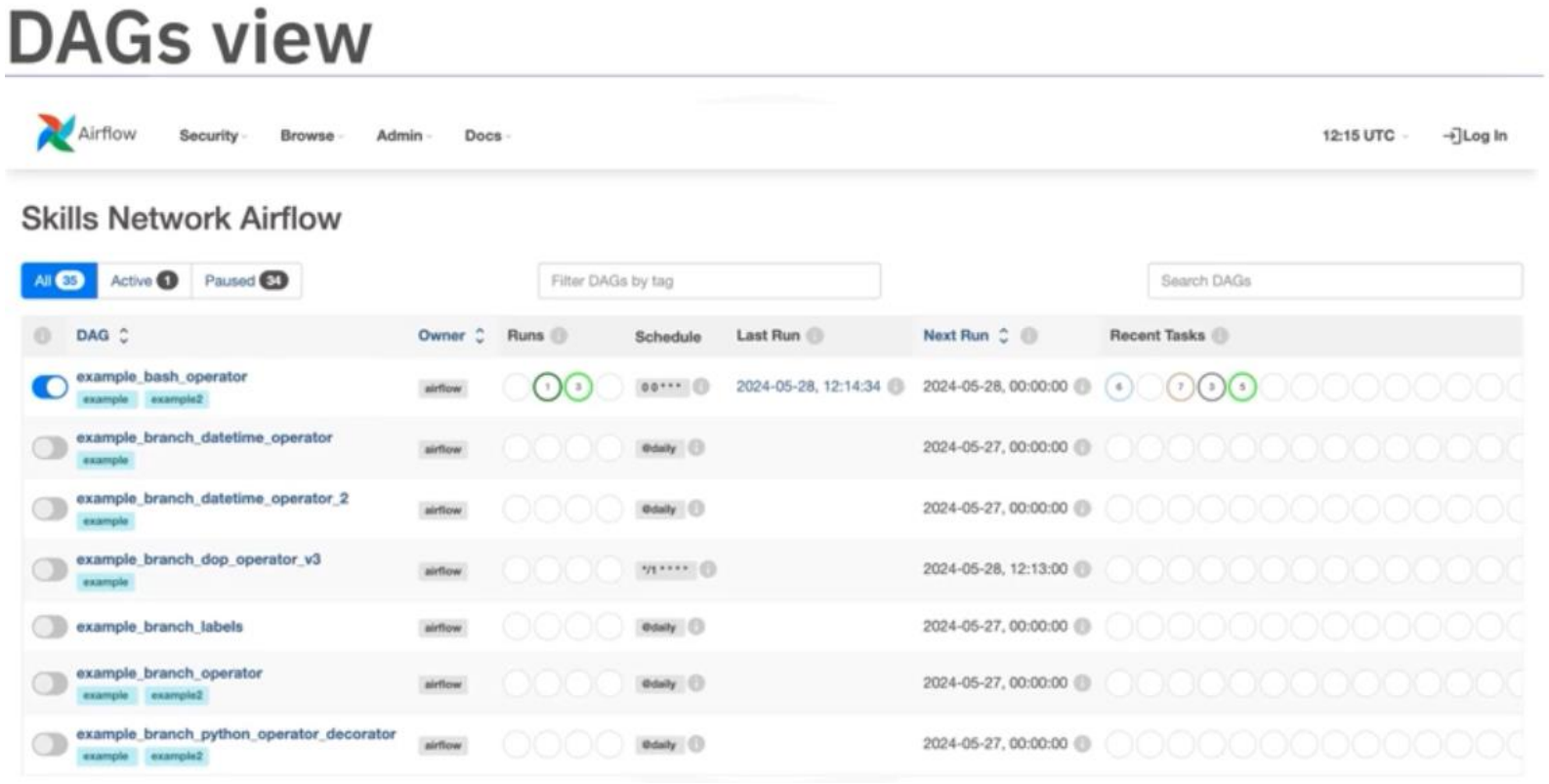
- **Heart of Airflow orchestration**
- Continuously monitors DAGs in the DAG directory
- Decides **what tasks need to run** and when
- Places runnable tasks into the **queue**
- Works with the **Executor** to distribute tasks to workers
- Supports **scheduling intervals, cron expressions, event triggers**

Advantages of DAGs

- Expressed as Python code → maintainable & explicit
- Versionable → trackable in Git
- Collaborative → multiple developers can contribute
- Testable → revisions validated by unit tests

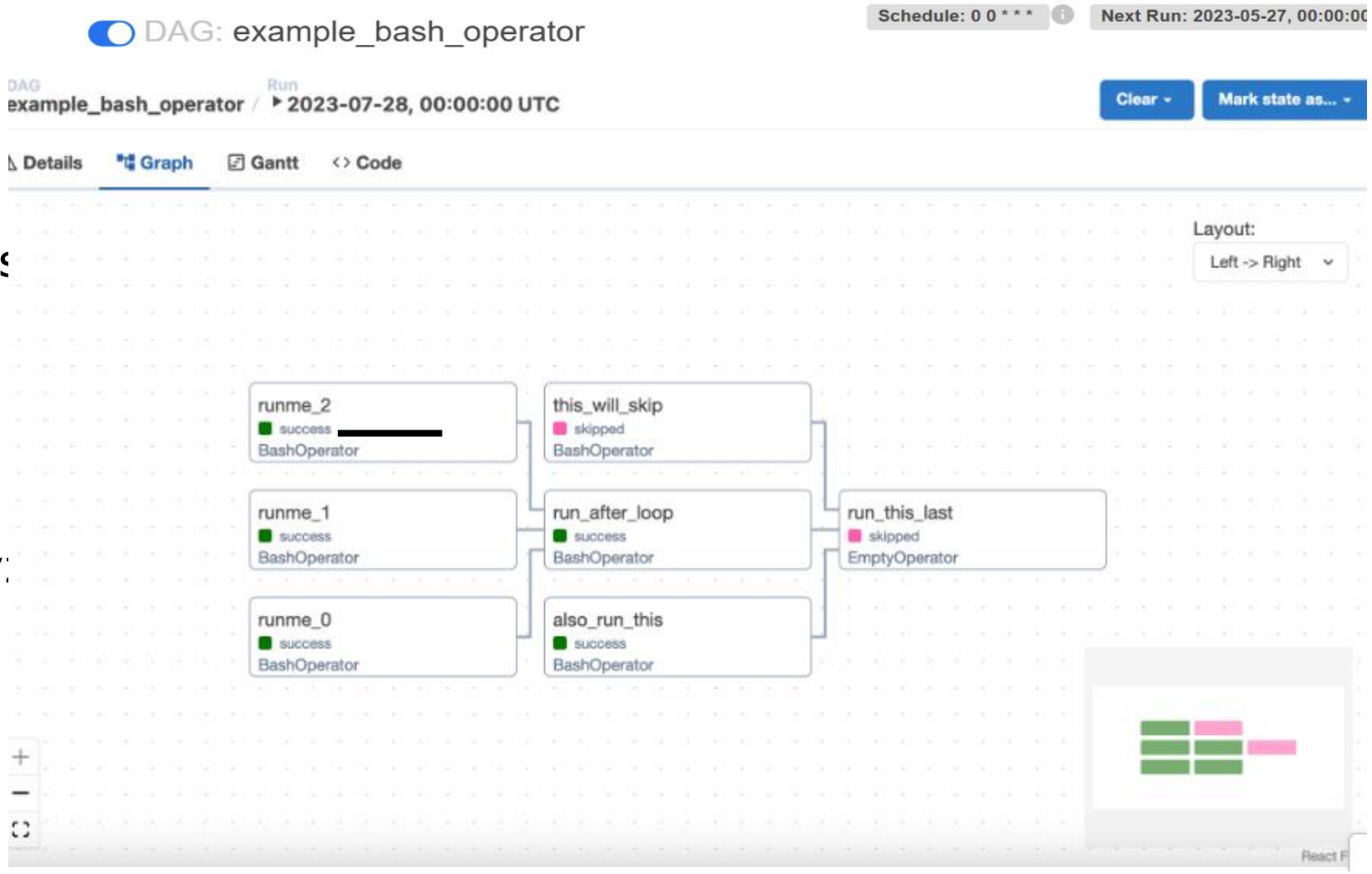
Airflow UI – DAGs View

- Table of DAGs (name, owner, schedule, status)
- Pause/resume toggle
- Quick links to DAG details



Visualizing DAGs

- Grid View → timeline of runs
- Graph View → tasks & dependencies
- Code View → inspect DAG Python file
- Task Duration View → analy runtimes



HSLU

Building a DAG: Key Blocks

- Imports
- DAG arguments dict (retries, owner, start_date, etc.)
- DAG definition (id, description, schedule)
- Task definitions (operators)
- Task pipeline (dependencies)

Example: simple_example_DAG.py

- Task1: BashOperator → print a message and the current time to the task log.
- Task2: BashOperator → the same as task 1, running after task 1.
- Dependency: task1 >> task2
- Scheduled to run every 5 seconds

DAG Arguments

- A python dictionary

Number of times it should try again if it fails.

The time to wait between tries.

```
default_args = {
    "owner": "me",
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}
```

DAG Definition

```
with DAG(
    dag_id="simple_tasks",
    description="Two simple Bash tasks chained: task_1 -> task_2",
    default_args=default_args,
    start_date=days_ago(1),       # not in the future
    schedule_interval="* * * * *",# every minute (change as you like)
    catchup=False,
    tags=["example"],
```

Task Definitions

```
task_1 = BashOperator(
    task_id="task_1",
    bash_command='echo "Running TASK 1: $(date)"'
)

task_2 = BashOperator(
    task_id="task_2",
    bash_command='echo "Running TASK 2 after TASK 1: $(date)"'
)
```

Airflow Pipeline Script

- Python library imports

```
from datetime import datetime, timedelta
from airflow.models import DAG
from airflow.operators.bash_operator import BashOperator
```

Airflow Logging

- Default: local log files per task attempt
- Production: logs to cloud storage
- Recommended: Elasticsearch or Splunk to index, search and analyze logfiles.

- Log files are organized as logs/dag_id=??/run_id=??/task_id=??/attempt=??